CPSY 1291 · RECITATION HANDOUT 9

Looking inside a network

Hooks, gradients with respect to the input, and how far to trust a saliency map

Fall 2026 · Week 9 · hold this after Lecture 13; Assignment 4 is due Tue 11/10

What this session is for

Last week you compared a network with a brain from the outside, by what could be read out of each. This week you open the network up.

The methods here are genuinely useful and genuinely less trustworthy than anything else in the course, and both halves of that sentence matter. A saliency map is easy to produce, looks compelling, and can be almost independent of what the model actually learned. So this session is arranged in pairs: here is how to compute it, and here is the check that tells you whether it means anything.

> **The one idea.** Every interpretability method answers a question of the form "what would change if…?" — if this pixel changed, if this unit were removed, if this feature were amplified. *Which* counterfactual a method asks is what distinguishes the methods, and reading a figure without knowing its counterfactual is how people over-read them.

1 Forward hooks, properly

A hook is a function a module calls with its own input and output every time it runs. It is how you get at activations in the middle of a model you did not write.

```
acts = {}

def save(name):
   def hook(module, inputs, output):
      acts[name] = output.detach()
   return hook

handles = [
   net.features[3].register_forward_hook(save('conv2')),
   net.features[8].register_forward_hook(save('conv4')),
]

with torch.no_grad():
   net(x) # the hooks fire during this call

for h in handles:
   h.remove() # always
```

> **The three ways hooks go wrong.** (i) **No `.detach()`** — you keep the autograd graph for every saved tensor and run out of memory on the third batch. (ii) **No `.remove()`** — the hook keeps firing on every later forward pass and silently overwrites your stored activations with whatever ran last. (iii) **Reading `acts` before calling the model** — hooks fire during `net(x)`, not when you register them. The dictionary is empty until then.

For most layers you do not need a hook at all: `net.features[:9](x)` runs a prefix and returns its output directly. Reach for hooks when the layer you want is not a prefix, when you want several layers from one forward pass, or when the model's `forward` does something slicing cannot reproduce.

1.1 What comes out, and how to reduce it

A convolutional activation is (N, C, H, W). To compare it with anything you usually flatten it per stimulus:

```
A = acts['conv4'] # (N, C, H, W)
X = A.flatten(1).numpy() # (N, C*H*W) -- for RSA, decoding
chan = A.mean(dim=(2, 3)).numpy() # (N, C) -- channel means
```

The choice between them is not cosmetic. Flattening keeps spatial information and gives you tens of thousands of features; averaging over space discards position and gives you one number per feature map. An RSA score computed on the two is answering two different questions, and papers differ on which they use.

2 Gradients with respect to the input

Training asks how the loss changes with the *weights*. Interpretability asks how the output changes with the *input*. Same machinery, different variable.

```
x = x.clone().requires_grad_(True) # track gradients for the INPUT
logits = net(x)
score = logits[0, class_idx] # one scalar: this class's logit
net.zero_grad()
score.backward()
g = x.grad[0] # (3, H, W)
```

Three details that decide whether this works:

- **Differentiate the logit, not the softmax probability.** A probability can be near 1 with a flat gradient simply because the softmax has saturated, and you get a blank map for a confident prediction.
- **Do not wrap this in `torch.no_grad()`.** The graph is the point here.
- **`net.zero_grad()`** first, or leftover gradients from a previous call are added in.

3 Saliency, and the checks it needs

3.1 The map

```
sal = g.abs().max(dim=0).values # (H, W): max over color channels
plt.imshow(sal, cmap='hot')
```

Two conventions worth understanding rather than copying. The **absolute value** is taken because a pixel that would strongly *decrease* the score is also evidence the model is using it; if you care about direction, keep the sign and use a diverging colormap. The **max over channels** is one choice among several — summing, or taking the norm, are equally defensible and give visibly different pictures.

> **What a saliency map actually claims.** That an infinitesimal change to this pixel would change this logit by this much, *at this exact input*. It is a local linear approximation. It does not say the model "looked at" the region, it does not say the region is sufficient, and it does not say the region is necessary.

3.2 The sanity check that should always accompany it

> **Randomize the weights and recompute.** Replace the network's parameters with random values, layer by layer from the top, and produce the saliency map again. If it still highlights the object, the map is being produced by the image and the architecture rather than by anything the model learned — and it is evidence about edges, not about the model. This check fails for several published methods.

A cheaper, weaker check: change the input by an imperceptible amount and recompute. Saliency maps are notoriously unstable to small perturbations, and seeing that instability for yourself is worth more than reading about it.

3.3 Smoothing, and what it costs

Averaging the maps from many noisy copies of the input ("SmoothGrad") produces a far cleaner picture:

```
maps = []
for _ in range(50):
   xn = x + 0.1 * torch.randn_like(x)
   maps.append(saliency(net, xn, class_idx))
smooth = torch.stack(maps).mean(0)
```

It is more stable and more interpretable. It also no longer answers the original question — it describes an average over a neighborhood of inputs rather than the gradient at your image — and it introduces a noise level you chose. Both facts belong in the caption.

4 Grad-CAM, in ten lines

Saliency is per-pixel and noisy. Grad-CAM works at the level of a convolutional layer's feature maps: weight each channel by how much increasing it would raise the score, then sum.

```
A = acts['last_conv'] # (1, C, h, w) -- from a forward hook
dA = grads['last_conv'] # (1, C, h, w) -- from a backward hook
w = dA.mean(dim=(2, 3), keepdim=True) # (1, C, 1, 1): importance per channel
cam = torch.relu((w * A).sum(1))[0] # (h, w)
cam = torch.nn.functional.interpolate(
      cam[None, None], size=x.shape[-2:], mode='bilinear')[0, 0]
```

The ReLU keeps only evidence *for* the class. The output is coarse — 7×7 for a typical last convolutional layer, upsampled to the image size — which is honest: that is genuinely the resolution at which this layer represents anything.

5 Maximally activating patches

The oldest method, and often the most informative.

```
resp = acts['conv4'][:, unit].amax(dim=(1, 2)) # peak response per image
top = resp.argsort(descending=True)[:9]
```

Then crop each top image around the location of that unit's peak, using the layer's receptive field size, and display the nine crops in a grid. What you are looking for is what the nine have in common.

> **The bias built into this method.** It shows you what drives the unit *within your dataset*. A unit whose true preference does not occur in your images will still return nine patches, and they will still have something in common — because you sorted for it. The check is to look at the response values as well as the images: if the top nine responses are barely above the median, you are looking at noise arranged by a sort.

Units are also frequently **polysemantic**: the top nine patches contain two or three unrelated things. That is a real property of trained networks, not a failure of the method, and it is the motivation for sparse-autoencoder features in Lecture 13.

6 Feature visualization

Instead of searching a dataset, optimize an input to maximize a unit:

```
x = torch.randn(1, 3, 224, 224, requires_grad=True)
opt = torch.optim.Adam([x], lr=0.05)
for _ in range(256):
   opt.zero_grad()
   (-net.features[:9](x)[0, unit].mean()).backward() # maximize -> minimize -aG
```

```
    opt.step()
```

Run exactly this and you get high-frequency noise that drives the unit hard and resembles nothing. Every published feature visualization uses regularization to avoid it: jittering the image between steps, blurring, penalizing total variation, or optimizing in a decorrelated color space.

> **The honest caveat.** Those regularizers are chosen because they produce pictures people find interpretable. That makes feature visualization partly a statement about the model and partly a statement about the prior you imposed, and the images are not evidence in the way a measurement is.

7 Comparing a model's attribution with a human's

If you have human attention maps for the same images — from eye tracking, or a bubbles-style paradigm, or a click-based one — you can score the agreement. The mechanics are Recitation 8's, one image at a time:

1. Resize both maps to the same resolution.
2. Rank-correlate them, using Spearman on the flattened maps: the two are on unrelated scales, and only their ordering is comparable.
3. Average over images to get one score per model.
4. **Compute the noise ceiling**: split your human participants in half and correlate one half's maps with the other's. Models are compared against that, not against 1.

The last step is where this analysis usually goes wrong, and it is the same error as last week's: without the ceiling, a model scoring 0.3 looks like a failure when it may be near the achievable maximum.

8 Exercises

1. You register a hook, then immediately print your `acts` dictionary. It is empty. Why?

2. Your saliency map is entirely black for an image the model classifies correctly with 99.8% confidence. What did you differentiate, and what should you have differentiated?

3. You randomize a network's weights and its saliency map still outlines the object. What have you learned?

4. A unit's top nine patches all contain grass. Give two different conclusions, and the number you would check to choose between them.

5. You run the feature-visualization loop above with no regularization. Describe the image, and say why it is not a bug.

6. A model's attribution maps correlate 0.28 with human maps. Is that good?

Answers

1. Hooks fire during `net(x)`, not when they are registered. You have not run a forward pass yet.

2. You differentiated the softmax probability, which has saturated near 1 and therefore has a nearly flat gradient. Differentiate the raw logit for that class instead.

3. That the map is being produced by the image and the architecture rather than by anything the model learned. The figure is evidence about edges, and any claim it was supporting about the model's strategy is unsupported.

4. (i) The unit detects grass texture. (ii) The unit detects something that merely co-occurs with grass in your dataset — a shortcut. Check the response *values*: if the top nine are far above the median the unit is genuinely selective for something present in all nine; if they are barely above it, the sort produced the commonality. To separate (i) from (ii) properly you need images with the feature and without the grass.

5. High-frequency noise that drives the unit very hard and resembles nothing. It is not a bug: the optimizer is doing exactly what you asked, and there is no reason the input maximizing a unit should be a natural image. Every interpretable feature visualization has a prior imposed on it.

6. Unanswerable without the noise ceiling. Split the human participants in half and correlate one half's maps with the other's; if that is 0.35, then 0.28 is excellent. If it is 0.9, it is poor.

Cheat sheet

You want	You write
activations from a prefix	`net.features[:9](x)`
several layers at once	`register_forward_hook`, then `.remove()`
conv activations as features	`A.flatten(1)` or `A.mean(dim=(2,3))`
gradient w.r.t. the input	`x.requires_grad_(True)`; `logit.backward()`
a saliency map	`x.grad[0].abs().max(0).values`
the sanity check	randomize the weights and recompute
a smoother map	average over 50 noisy copies of the input
Grad-CAM	`relu((dA.mean((2,3)) * A).sum(1))`
top-k images for a unit	`resp.argsort(descending=True)[:9]`
agreement with human maps	Spearman per image, then a split-half ceiling